

## JU PS CBT-11

Total Marks: 50 | Duration: 60 | Total Questions: 2

## Module: Coding

Marks: 50 | Duration: 0

## Question 1:

Marks: +20 -0

## Unique Elements in Sorted Array

Given a sorted array `arr[]` of size `n`, the goal is to rearrange the array so that all distinct elements appear at the beginning in sorted order. Additionally, return the length of this distinct sorted subarray.

**Note:** The elements after the distinct ones can be in any order and hold any value, as they don't affect the result.

## Examples:

**Input:** `arr[] = [2, 2, 2, 2, 2]`

**Output:** `[2]`

**Explanation:** All the elements are 2, So only keep one instance of 2.

**Input:** `arr[] = [1, 2, 2, 3, 4, 4, 4, 5, 5]`

**Output:** `[1, 2, 3, 4, 5]`

**Input:** `arr[] = [1, 2, 3]`

**Output:** `[1, 2, 3]`

**Explanation :** No change as all elements are distinct.

## Test Cases:

| # | Input                  | Expected Output |
|---|------------------------|-----------------|
| 1 | 9<br>1 2 2 3 4 4 4 5 5 | 5<br>1 2 3 4 5  |
| 2 | 5<br>2 2 2 2 2         | 1<br>2          |

**Solution:**

```

import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        StringTokenizer st;

        // Read n
        String line = br.readLine();
        if (line == null || line.trim().isEmpty()) {
            return; // no input
        }
        int n = Integer.parseInt(line.trim());

        int[] arr = new int[n];
        int idx = 0;
        while (idx < n) {
            String l = br.readLine();
            if (l == null) break;
            st = new StringTokenizer(l);
            while (st.hasMoreTokens() && idx < n) {
                arr[idx++] = Integer.parseInt(st.nextToken());
            }
        }

        if (n == 0) {
            // As per constraints, this shouldn't usually happen, but handle
gracefully
            System.out.println(0);
            return;
        }

        int uniqueLen = removeDuplicatesInPlace(arr);

        // Output: first line is length of distinct prefix
        // second line is the distinct elements in sorted order
        System.out.println(uniqueLen);
        StringBuilder sb = new StringBuilder();

```

```

        for (int i = 0; i < uniqueLen; i++) {
            if (i > 0) sb.append(' ');
            sb.append(arr[i]);
        }
        System.out.println(sb.toString());
    }

    // Rearranges the array so that the first part contains all unique elements
    // in sorted order (given original array is sorted). Returns the count of
    // unique elements. Operates in-place with O(1) extra space.
    private static int removeDuplicatesInPlace(int[] arr) {
        int n = arr.length;
        if (n == 0) return 0;
        int write = 1; // index to write next unique element
        for (int read = 1; read < n; read++) {
            if (arr[read] != arr[write - 1]) {
                arr[write] = arr[read];
                write++;
            }
        }
        return write;
    }
}

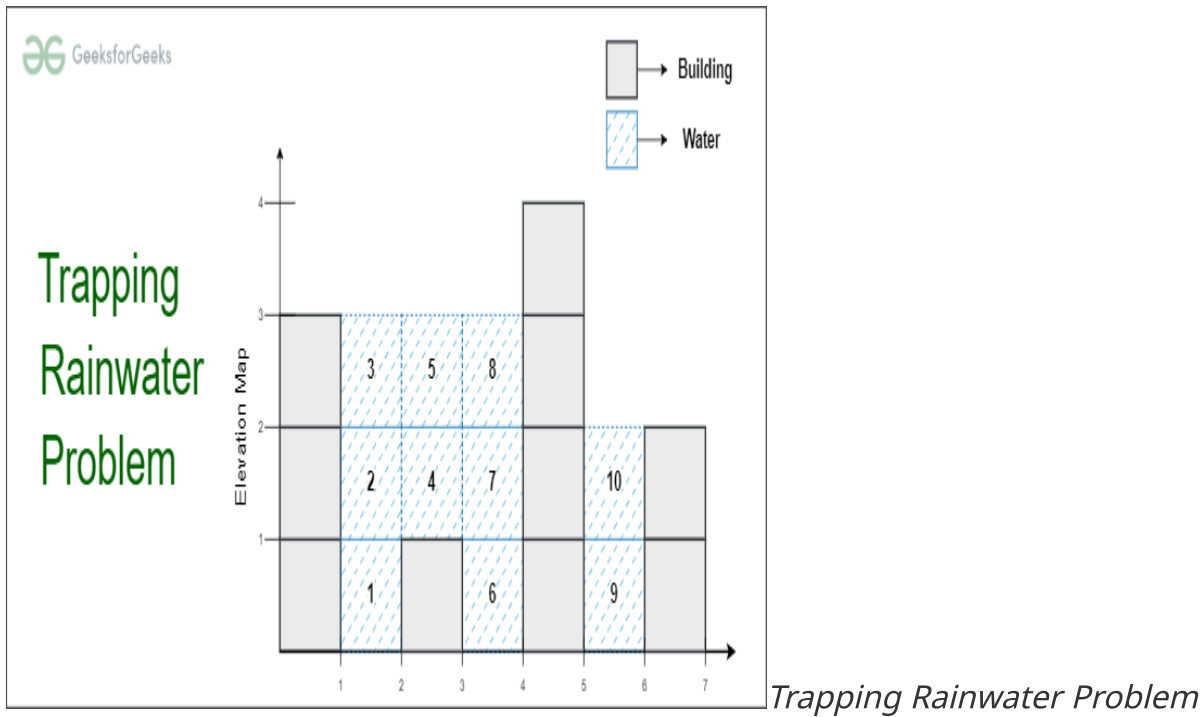
```

## Question 2:

Marks: +30 -0

### Trapping Rain Water Problem

Given an array `arr[]` of size `n` consisting of non-negative integers, where each element represents the height of a bar in an elevation map and the width of each bar is 1, determine the total amount of water that can be trapped between the bars after it rains.



### Examples:

**Input:**  $arr[] = [3, 0, 1, 0, 4, 0, 2]$

**Output:** 10

**Explanation:** The expected rainwater to be trapped is shown in the above image.

**Input:**  $arr[] = [3, 0, 2, 0, 4]$

**Output:** 7

**Explanation:** We trap  $0 + 3 + 1 + 3 + 0 = 7$  units.

**Input:**  $arr[] = [1, 2, 3, 4]$

**Output:** 0

**Explanation:** We cannot trap water as there is no height bound on both sides

### Test Cases:

| # | Input | Expected Output |
|---|-------|-----------------|
|---|-------|-----------------|

|   |                    |    |
|---|--------------------|----|
| 1 | 7<br>3 0 1 0 4 0 2 | 10 |
| 2 | 5<br>3 0 2 0 4     | 7  |

**Solution:**

```
import sys

def trap_rain_water(heights):
    n = len(heights)
    if n == 0:
        return 0

    left = 0
    right = n - 1
    left_max = 0
    right_max = 0
    water = 0

    while left <= right:
        if heights[left] <= heights[right]:
            if heights[left] >= left_max:
                left_max = heights[left]
            else:
                water += left_max - heights[left]
            left += 1
        else:
            if heights[right] >= right_max:
                right_max = heights[right]
            else:
                water += right_max - heights[right]
            right -= 1

    return water

if __name__ == "__main__":
    data = sys.stdin.read().strip().split()
    if not data:
        print(0)
        sys.exit(0)

    n = int(data[0])
    arr = []
    if n > 0:
        # Take next n integers as the elevation map
        for i in range(n):
```

```
        if 1 + i < len(data):
            arr.append(int(data[1 + i]))
        else:
            arr.append(0)
    result = trap_rain_water(arr)
    print(result)
```

SKELO